

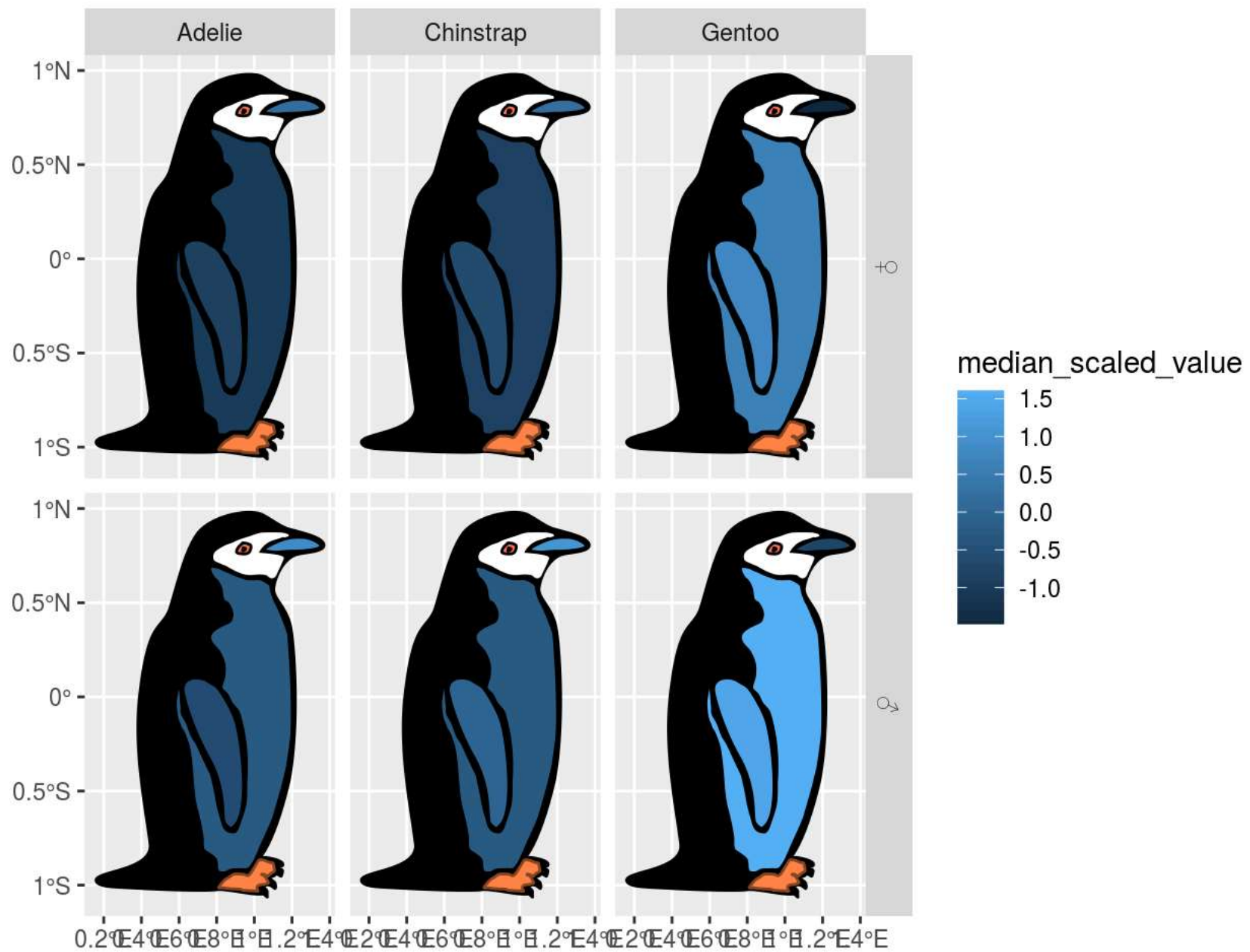
```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: sns.get_dataset_names()
```

```
Out[2]: ['anagrams',
         'anscombe',
         'attention',
         'brain_networks',
         'car_crashes',
         'diamonds',
         'dots',
         'dowjones',
         'exercise',
         'flights',
         'fmri',
         'geyser',
         'glue',
         'healthexp',
         'iris',
         'mpg',
         'penguins',
         'planets',
         'seaice',
         'taxi',
         'tips',
         'titanic',
         'anagrams',
         'anagrams',
         'anscombe',
         'anscombe',
         'attention',
         'attention',
         'brain_networks',
         'brain_networks',
         'car_crashes',
         'car_crashes',
         'diamonds',
         'diamonds',
         'dots',
         'dots',
         'dowjones',
         'dowjones',
         'exercise',
         'exercise',
         'flights',
```

'flights',
'fmri',
'fmri',
'geyser',
'geyser',
'glue',
'glue',
'healthexp',
'healthexp',
'iris',
'iris',
'mpg',
'mpg',
'penguins',
'penguins',
'planets',
'planets',
'seaice',
'seaice',
'taxi',
'taxi',
'tips',
'tips',
'titanic',
'titanic',
'anagrams',
'anscombe',
'attention',
'brain_networks',
'car_crashes',
'diamonds',
'dots',
'dowjones',
'exercise',
'flights',
'fmri',
'geyser',
'glue',
'healthexp',
'iris',
'mpg',
'penguins',

```
'planets',  
'seaice',  
'taxi',  
'tips',  
'titanic']
```



```
In [3]: df = sns.load_dataset('penguins')
df
```

```
Out[3]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female
...	...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	Male
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	Female
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0	Male

344 rows × 7 columns

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species                344 non-null    object
1   island                 344 non-null    object
2   bill_length_mm         342 non-null    float64
3   bill_depth_mm          342 non-null    float64
4   flipper_length_mm      342 non-null    float64
5   body_mass_g            342 non-null    float64
6   sex                    333 non-null    object
dtypes: float64(4), object(3)
memory usage: 18.9+ KB
```

```
In [5]: df.describe()
```

```
Out[5]:
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.000000	342.000000	342.000000	342.000000
mean	43.921930	17.151170	200.915205	4201.754386
std	5.459584	1.974793	14.061714	801.954536
min	32.100000	13.100000	172.000000	2700.000000
25%	39.225000	15.600000	190.000000	3550.000000
50%	44.450000	17.300000	197.000000	4050.000000
75%	48.500000	18.700000	213.000000	4750.000000
max	59.600000	21.500000	231.000000	6300.000000

```
In [6]: df['bill_length_mm']
```

```
Out[6]:
```

0	39.1
1	39.5
2	40.3
3	NaN
4	36.7
...	
339	NaN
340	46.8
341	50.4
342	45.2
343	49.9

Name: bill_length_mm, Length: 344, dtype: float64

```
In [7]: df['bill_length_mm'].quantile(0.25)
```

```
Out[7]: 39.225
```

```
In [ ]:
```

```
In [8]: df['bill_length_mm'].quantile(0.75)
```

```
Out[8]: 48.5
```

```
In [ ]:
```

Pentiles

```
In [9]: df.describe(percentiles=[0.2, 0.4, 0.6, 0.8, 1.0])
```

```
Out[9]:
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.000000	342.000000	342.000000	342.000000
mean	43.921930	17.151170	200.915205	4201.754386
std	5.459584	1.974793	14.061714	801.954536
min	32.100000	13.100000	172.000000	2700.000000
20%	38.340000	15.000000	188.000000	3475.000000
40%	42.000000	16.800000	194.000000	3800.000000
50%	44.450000	17.300000	197.000000	4050.000000
60%	46.000000	17.900000	203.000000	4300.000000
80%	49.380000	18.900000	215.000000	4950.000000
100%	59.600000	21.500000	231.000000	6300.000000
max	59.600000	21.500000	231.000000	6300.000000

```
In [10]: df.describe(percentiles=[0.2, 0.4, 0.6, 0.8])
```

```
Out[10]:
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.000000	342.000000	342.000000	342.000000
mean	43.921930	17.151170	200.915205	4201.754386
std	5.459584	1.974793	14.061714	801.954536
min	32.100000	13.100000	172.000000	2700.000000
20%	38.340000	15.000000	188.000000	3475.000000
40%	42.000000	16.800000	194.000000	3800.000000
50%	44.450000	17.300000	197.000000	4050.000000
60%	46.000000	17.900000	203.000000	4300.000000
80%	49.380000	18.900000	215.000000	4950.000000
max	59.600000	21.500000	231.000000	6300.000000

```
In [ ]:
```

Deciles

```
In [11]: df.describe(percentiles=[0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])
```

```
Out[11]:
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.000000	342.000000	342.000000	342.000000
mean	43.921930	17.151170	200.915205	4201.754386
std	5.459584	1.974793	14.061714	801.954536
min	32.100000	13.100000	172.000000	2700.000000
10%	36.600000	14.300000	185.000000	3300.000000
20%	38.340000	15.000000	188.000000	3475.000000
30%	40.200000	15.930000	191.000000	3650.000000
40%	42.000000	16.800000	194.000000	3800.000000
50%	44.450000	17.300000	197.000000	4050.000000
60%	46.000000	17.900000	203.000000	4300.000000
70%	47.370000	18.500000	210.000000	4650.000000
80%	49.380000	18.900000	215.000000	4950.000000
90%	50.800000	19.500000	220.900000	5400.000000
max	59.600000	21.500000	231.000000	6300.000000

```
In [ ]:
```

Sampling Techniques

In [12]: df

Out[12]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female
...	...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	Male
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	Female
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0	Male

344 rows × 7 columns

In []:

Simple Random Sampling

```
In [13]: n = 10

sample_df = df.sample(n)
print('Simple Random Sampling: ')
sample_df
```

Simple Random Sampling:

```
Out[13]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
262	Gentoo	Biscoe	45.3	13.7	210.0	4300.0	Female
132	Adelie	Dream	36.8	18.5	193.0	3500.0	Female
291	Gentoo	Biscoe	46.4	15.6	221.0	5000.0	Male
159	Chinstrap	Dream	51.3	18.2	197.0	3750.0	Male
115	Adelie	Biscoe	42.7	18.3	196.0	4075.0	Male
116	Adelie	Torgersen	38.6	17.0	188.0	2900.0	Female
328	Gentoo	Biscoe	43.3	14.0	208.0	4575.0	Female
94	Adelie	Dream	36.2	17.3	187.0	3300.0	Female
65	Adelie	Biscoe	41.6	18.0	192.0	3950.0	Male
189	Chinstrap	Dream	52.0	20.7	210.0	4800.0	Male

```
In [ ]:
```

Systematic Sampling

```
In [14]: n = 15  
new_df = df.iloc[:n]  
new_df
```

Out[14]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
15	Adelie	Torgersen	36.6	17.8	185.0	3700.0	Female
30	Adelie	Dream	39.5	16.7	178.0	3250.0	Female
45	Adelie	Dream	39.6	18.8	190.0	4600.0	Male
60	Adelie	Biscoe	35.7	16.9	185.0	3150.0	Female
75	Adelie	Torgersen	42.8	18.5	195.0	4250.0	Male
90	Adelie	Dream	35.7	18.0	202.0	3550.0	Female
105	Adelie	Biscoe	39.7	18.9	184.0	3550.0	Male
120	Adelie	Torgersen	36.2	17.2	187.0	3150.0	Female
135	Adelie	Dream	41.1	17.5	190.0	3900.0	Male
150	Adelie	Dream	36.0	17.1	187.0	3700.0	Female
165	Chinstrap	Dream	52.0	18.1	201.0	4050.0	Male
180	Chinstrap	Dream	46.4	17.8	191.0	3700.0	Female
195	Chinstrap	Dream	45.5	17.0	196.0	3500.0	Female
210	Chinstrap	Dream	50.2	18.8	202.0	3800.0	Male
225	Gentoo	Biscoe	46.5	13.5	210.0	4550.0	Female
240	Gentoo	Biscoe	50.2	14.3	218.0	5700.0	Male
255	Gentoo	Biscoe	48.4	16.3	220.0	5400.0	Male
270	Gentoo	Biscoe	46.6	14.2	210.0	4850.0	Female
285	Gentoo	Biscoe	49.8	16.8	230.0	5700.0	Male
300	Gentoo	Biscoe	49.1	14.5	212.0	4625.0	Female
315	Gentoo	Biscoe	50.8	15.7	226.0	5200.0	Male
330	Gentoo	Biscoe	50.5	15.2	216.0	5000.0	Female

```
In [15]: n = 5

new_df = df.iloc[:n]
new_df
```

```
Out[15]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
5	Adelie	Torgersen	39.3	20.6	190.0	3650.0	Male
10	Adelie	Torgersen	37.8	17.1	186.0	3300.0	NaN
15	Adelie	Torgersen	36.6	17.8	185.0	3700.0	Female
20	Adelie	Biscoe	37.8	18.3	174.0	3400.0	Female
...	...	...	...	...	...	...	...
320	Gentoo	Biscoe	48.5	15.0	219.0	4850.0	Female
325	Gentoo	Biscoe	46.8	16.1	215.0	5500.0	Male
330	Gentoo	Biscoe	50.5	15.2	216.0	5000.0	Female
335	Gentoo	Biscoe	55.1	16.0	230.0	5850.0	Male
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female

69 rows × 7 columns

```
In [ ]:
```

Startified Sampling

```
In [34]: stratified = df.groupby('species').apply(lambda x: x.sample(min(len(x), 5)))
stratified
```

Out[34]:

		species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
species								
Adelie	61	Adelie	Biscoe	41.3	21.1	195.0	4400.0	Male
	68	Adelie	Torgersen	35.9	16.6	190.0	3050.0	Female
	117	Adelie	Torgersen	37.3	20.5	199.0	3775.0	Male
	113	Adelie	Biscoe	42.2	19.5	197.0	4275.0	Male
	42	Adelie	Dream	36.0	18.5	186.0	3100.0	Female
Chinstrap	212	Chinstrap	Dream	51.9	19.5	206.0	3950.0	Male
	164	Chinstrap	Dream	47.0	17.3	185.0	3700.0	Female
	210	Chinstrap	Dream	50.2	18.8	202.0	3800.0	Male
	170	Chinstrap	Dream	46.4	18.6	190.0	3450.0	Female
	172	Chinstrap	Dream	42.4	17.3	181.0	3600.0	Female
Gentoo	263	Gentoo	Biscoe	49.6	15.0	216.0	4750.0	Male
	313	Gentoo	Biscoe	49.5	16.1	224.0	5650.0	Male
	244	Gentoo	Biscoe	42.9	13.1	215.0	5000.0	Female
	254	Gentoo	Biscoe	49.1	14.8	220.0	5150.0	Female
	260	Gentoo	Biscoe	42.7	13.7	208.0	3950.0	Female

```
In [36]: stratified = df.groupby('species', group_keys=False).apply(lambda x: x.sample(min(len(x), 5)))
stratified
```

```
Out[36]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
48	Adelie	Dream	36.0	17.9	190.0	3450.0	Female
109	Adelie	Biscoe	43.2	19.0	197.0	4775.0	Male
97	Adelie	Dream	40.3	18.5	196.0	4350.0	Male
151	Adelie	Dream	41.5	18.5	201.0	4000.0	Male
202	Chinstrap	Dream	48.1	16.4	199.0	3325.0	Female
219	Chinstrap	Dream	50.2	18.7	198.0	3775.0	Female
190	Chinstrap	Dream	46.9	16.6	192.0	2700.0	Female
172	Chinstrap	Dream	42.4	17.3	181.0	3600.0	Female
196	Chinstrap	Dream	50.9	17.9	196.0	3675.0	Female
305	Gentoo	Biscoe	50.8	17.3	228.0	5600.0	Male
260	Gentoo	Biscoe	42.7	13.7	208.0	3950.0	Female
316	Gentoo	Biscoe	49.4	15.8	216.0	4925.0	Male
271	Gentoo	Biscoe	48.5	14.1	220.0	5300.0	Male
279	Gentoo	Biscoe	50.4	15.3	224.0	5550.0	Male

```
In [ ]:
```

Cluster Sampling

```
In [9]: clusters = df['species'].unique()
clusters
```

```
Out[9]: array(['Gentoo', 'Chinstrap'], dtype=object)
```

```
In [4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [7]: df = sns.load_dataset('penguins')
df
```

```
Out[7]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female
...	...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	Male
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	Female
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0	Male

344 rows × 7 columns

```
In [8]: clusters = df['species'].unique()

df = pd.concat([df[df['species'] == i].sample(2) for i in np.random.choice(clusters, size=2, replace=False)])
df
```

```
Out[8]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
313	Gentoo	Biscoe	49.5	16.1	224.0	5650.0	Male
321	Gentoo	Biscoe	55.9	17.0	228.0	5600.0	Male
194	Chinstrap	Dream	50.9	19.1	196.0	3550.0	Male
175	Chinstrap	Dream	50.6	19.4	193.0	3800.0	Male

```
In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [4]: df = sns.load_dataset('penguins')
df
```

```
Out[4]:
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female
...	...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	Male
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	Female
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0	Male

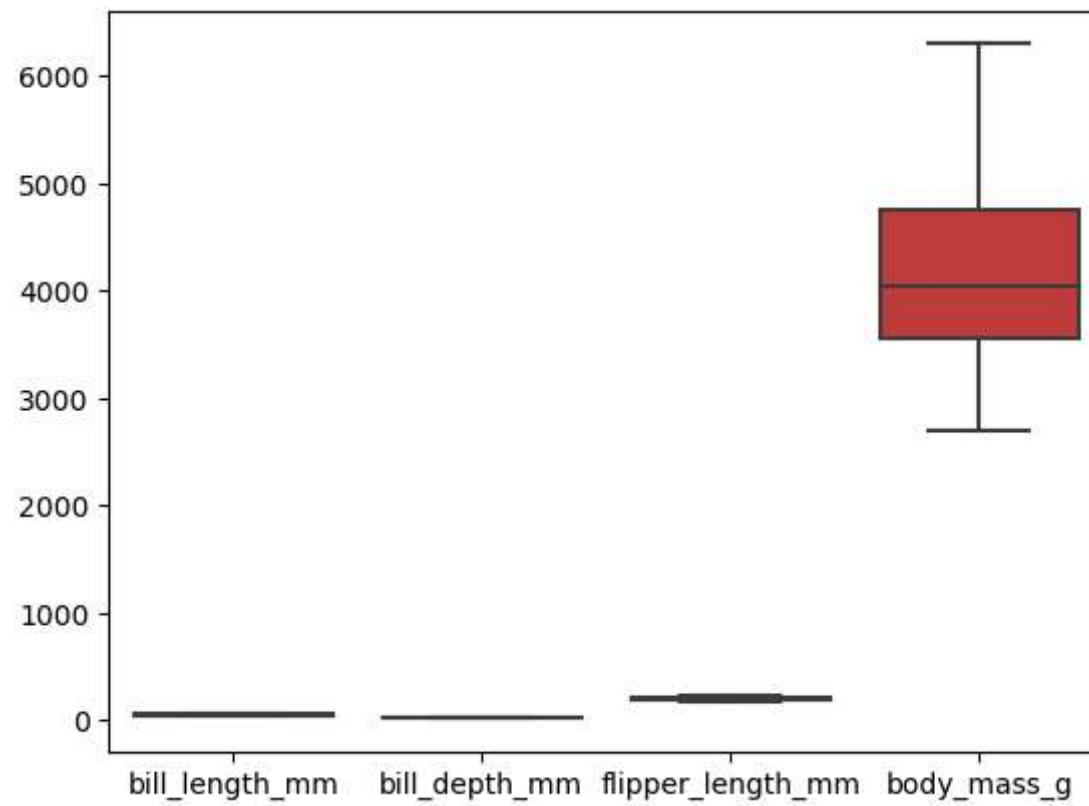
344 rows × 7 columns

```
In [ ]:
```

Outlier Detection using Boxplot

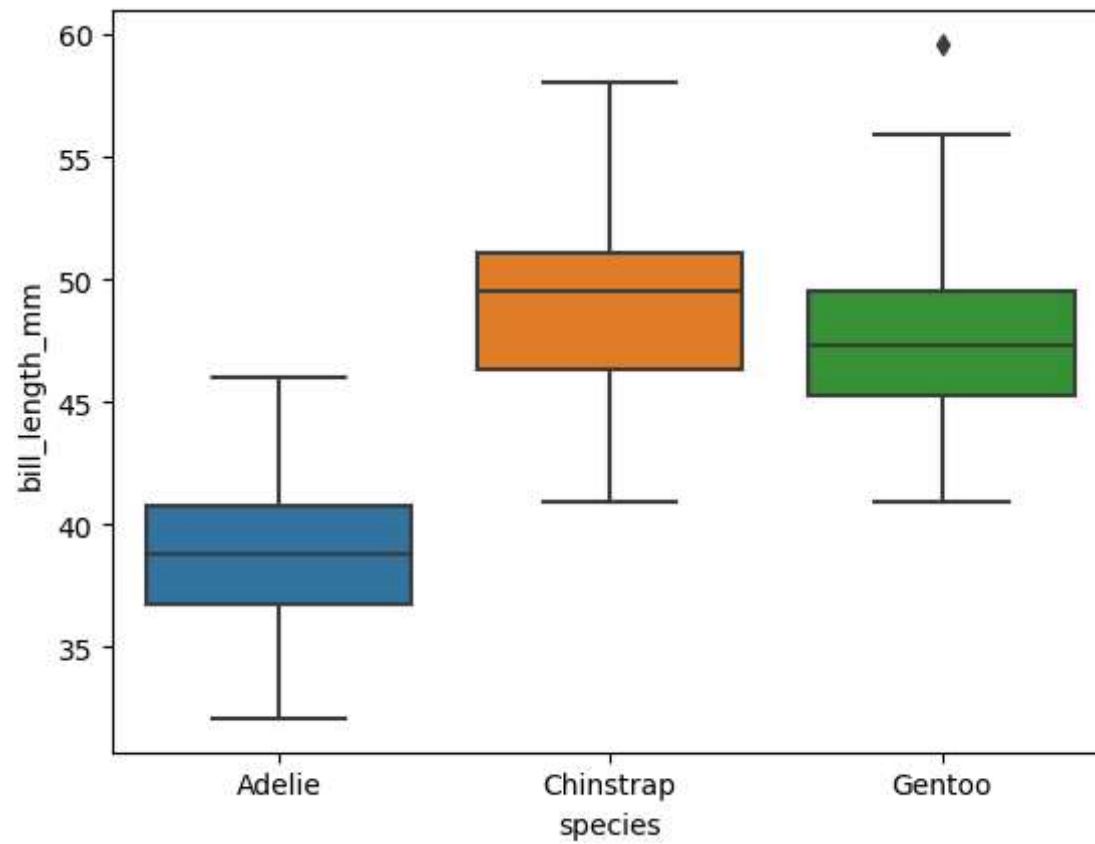
```
In [5]: sns.boxplot(df)
```

```
Out[5]: <Axes: >
```



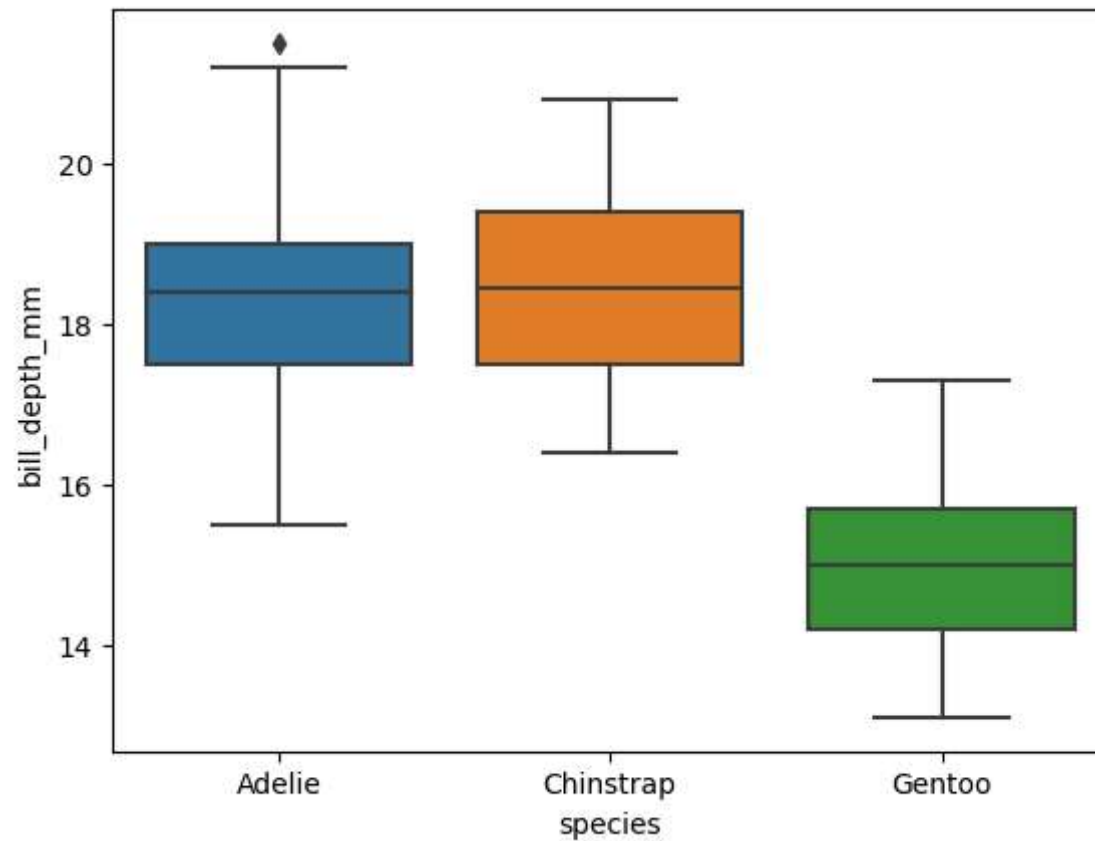
```
In [6]: sns.boxplot(x=df['species'], y=df['bill_length_mm'])
```

```
Out[6]: <Axes: xlabel='species', ylabel='bill_length_mm'>
```



```
In [8]: sns.boxplot(x=df['species'], y=df['bill_depth_mm'])
```

```
Out[8]: <Axes: xlabel='species', ylabel='bill_depth_mm'>
```



```
In [ ]:
```

Confidence Interval

$$CI = \bar{x} \pm z \frac{s}{\sqrt{n}}$$

CI = confidence interval

$\bar{x}$ = sample mean

z = confidence level value

s = sample standard deviation

n = sample size

Confidence Level to Z-Score

90%  1.645

95%  1.96

98%  2.33

99%  2.575

```
In [9]: df.bill_length_mm
```

```
Out[9]: 0      39.1  
1      39.5  
2      40.3  
3       NaN  
4      36.7  
      ...  
339     NaN  
340     46.8  
341     50.4  
342     45.2  
343     49.9  
Name: bill_length_mm, Length: 344, dtype: float64
```

```
In [14]: import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
In [15]: df = sns.load_dataset('penguins')
df
```

Out[15]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female
...	...	...	...	...	...	...	...
339	Gentoo	Biscoe	NaN	NaN	NaN	NaN	NaN
340	Gentoo	Biscoe	46.8	14.3	215.0	4850.0	Female
341	Gentoo	Biscoe	50.4	15.7	222.0	5750.0	Male
342	Gentoo	Biscoe	45.2	14.8	212.0	5200.0	Female
343	Gentoo	Biscoe	49.9	16.1	213.0	5400.0	Male

344 rows × 7 columns

```
In [10]: # population Mean
df.bill_length_mm.mean()
```

Out[10]: 43.9219298245614

```
In [16]: # sample data
sample = df.bill_length_mm.sample(50)
sample
```

```
Out[16]: 48      36.0
          58      36.5
          200     51.5
          148     36.0
          115     42.7
          38      37.6
          190     46.9
          85      41.3
          267     50.5
          30      39.5
          39      39.8
          133     37.5
          144     37.3
          153     50.0
          89      38.9
          255     48.4
          180     46.4
          270     46.6
          117     37.3
          55      41.4
          140     40.2
          3       NaN
          118     35.7
          7       39.2
          297     51.1
          275     45.0
          331     49.8
          62      37.6
          97      40.3
          113     42.2
          64      36.4
          59      37.6
          157     45.2
          182     40.9
          36      38.8
          338     47.2
          44      37.0
          322     47.2
          122     40.2
          199     49.0
          289     50.7
```

```
315    50.8
290    47.7
335    55.1
306    43.4
223    50.0
281    46.2
213    46.8
300    49.1
250    47.3
Name: bill_length_mm, dtype: float64
```

```
In [17]: import numpy as np
```

```
In [18]: # set a confidence level

confidence_level = 0.95 # 95 % confidence

Critical_value = 1.96
print(f'Critical value (z): {Critical_value}')

sample_mean = np.mean(sample)
print(f'sample mean (x): {sample_mean}')

sample_std = np.std(sample)
print(f'sample std (s): {sample_std}')

sample_size = len(sample)
print(f'sample size (n): {sample_size}')

Critical value (z): 1.96
sample mean (x): 43.465306122448986
sample std (s): 5.357944740762503
sample size (n): 50
```

$$CI = \bar{x} \pm z \frac{s}{\sqrt{n}}$$

```
In [19]: margin_of_error = Critical_value*(sample_std/np.sqrt(sample_size))
margin_of_error
```

```
Out[19]: 1.485146511291058
```

```
In [20]: lower_limit = sample_mean - margin_of_error
upper_limit = sample_mean + margin_of_error

print(f'confidence interval for bill length mm at {confidence_level*100}% confidence: ')
print()

print(f'Population Mean: {df.bill_length_mm.mean()}')
print(f'Sample Mean: {sample_mean}')
print(f'Margin of Error: {margin_of_error}')
print(f'Lower Limit: {lower_limit}')
print(f'Upper Limit: {upper_limit}')
```

confidence interval for bill length mm at 95.0% confidence:

Population Mean: 43.9219298245614
Sample Mean: 43.465306122448986
Margin of Error: 1.485146511291058
Lower Limit: 41.98015961115793
Upper Limit: 44.95045263374004

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: